

NHS Severe Patient Harm Forecasting

Authors: Charles Badger, Nonie Alexander, Jon Holdship

Organisation: Faculty

Date: June 2026

Introduction

We present the XGBoost based algorithm used to generate a 10-day forecast of the target variable, `estimated_avoidable_deaths`. At a high-level, we:

1. **Set-up, load data:** Load relevant forecasting and analysis functions. Loads and preprocesses the tabular NHS data.
2. **Feature engineering:** Feature engineer seasonal and calendar data.
3. **Feature and target lags:** Apply a range of lags to both the feature and target variables in order to create historical trends to learn from.
4. **Additional feature engineering:** Additional feature engineering focused rolling averages and trends of the target variable.
5. **Model training:** Train an XGBoost model specialised for each forecasted day using the lagged data sets.
6. **Model forecast:** Using the trained XGBoost models, we then forecast the next 10 days using the final day of training data.

Set-up, load data

We first load the necessary modules and scripts for the forecasting predictions. We then load the NHS tabular data and preprocess it using

`load_and_preprocess_data` to clean feature names, apply the midday operational boundary, aggregates values to compute daily means.

```
In [1]: import logging
import sys

import joblib
import numpy as np
import pandas as pd

sys.path.append("../src")
from diagnostic_toolkit import mse_metric
from feature_engineering import holiday_features, prepare_features, week1
from forecasting import create_features_and_targets
from preprocessing import load_and_preprocess_data
from xgboost_model import NHSForecaster

root = logging.getLogger()
root.handlers.clear()
```

```
logging.basicConfig(level=logging.INFO, format="%(name)s %(levelname)s %(
logger = logging.getLogger(__name__)

# Load data #
logger.info("Loading data.")
df = load_and_preprocess_data("../data/turingAI_forecasting_challenge_dat
df = df.reset_index()
logger.info("Data loaded.")
```

```
__main__ INFO Loading data.
preprocessing INFO Loading raw data from: ../data/turingAI_forecasting_cha
llenge_dataset.csv
preprocessing INFO Loading raw data from: ../data/turingAI_forecasting_cha
llenge_dataset.csv
preprocessing INFO Filtering dates and applying midday boundary...
preprocessing INFO Cleaning and standardising feature names...
preprocessing INFO Aggregating metrics to daily means...
preprocessing INFO Pivoting data to wide format...
preprocessing INFO Preprocessing complete. Final shape: (930, 218)
__main__ INFO Data loaded.
```

Feature engineering

We then select the features of interest, all of them in our case, and further engineer features to aid us in forecast training. We engineer two sets of calendar features via two functions:

- **weekly_yearly_features** : Quantifies weekly and yearly seasonal behavior with a sinusoidal function $\sin_{\text{weekly}} = \sin\left[\frac{2\pi \cdot \text{dayofweek}}{7}\right]$ and $\sin_{\text{yearly}} = \sin\left[\frac{2\pi \cdot \text{dayofyear}}{365.25}\right]$, and identifies the day of the year and flags weekends. Sinusoidal weekly and yearly features are used in preference calendar dummies to provide smooth, continuous seasonal signals that generalise across years without requiring a distinct marker for each day-of-week.
- **holiday_features** : Flags UK holidays using the **govuk_bank_holidays** module, identifies bridge days, and flags these as well as surrounding 3 days around each holiday. A Christmas / NYE season flag is added from December 10 through to January 10. The December 10–January 10 window is defined to capture the operationally distinct Christmas and New Year period, during which NHS staffing and demand patterns diverge substantially from standard bank holiday behaviour. Bridge days - workdays that fall between public holidays and the weekend - are also flagged to capture the tendency for operational pressure to accumulate across extended holiday periods, which a simple binary holiday flag would miss.

```
In [2]: forecasting_labels = list(df.columns.values)
target_label = ["estimated_avoidable_deaths"]
if forecasting_labels is None:
    forecasting_df = df[["midday_day"] + target_label]
    forecasting_exog = None
else:
    forecasting_df = df # df[["midday_day"] + forecasting_labels + target_label]
```

```

forecasting_exog = df[forecasting_labels]
forecasting_df["midday_day"] = pd.to_datetime(forecasting_df["midday_day"])

forecasting_df = weekly_yearly_features(forecasting_df)
forecasting_df = holiday_features(forecasting_df)

```

Feature and target lags

Creates day lags 0-14, 21 for the engineered features, and lags the target variable by 3-7, 10, 14, 21 days, all used for input `X` training data. Note, the target lags are all greater than or equal to 3 in order to observe the 3 day reporting delay of the target variable. As part of the `y` training data, a set of shifted target variables (shifts 1, 2, ..., 10) is generated as well labeled with `estimated_avoidable_deaths_step_[DAY]`.

A key constraint on this forecasting problem is a 3-day reporting delay on the target variable — the most recently available death value at forecast time is always `lag_3`. This means `lag_0` through `lag_2` of the target are never valid inputs, and the minimum valid autoregressive feature is `estimated_avoidable_deaths_lag3`. This constraint shapes the entire feature set: rolling statistics are computed from lags 3–7, and the direct forecasting approach — one model per horizon day — is used in part to avoid compounding this delay through recursive prediction.

```

In [3]: X, y = create_features_and_targets(
        forecasting_df,
        feature_lags=(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 21),
        target_lags=(3, 4, 5, 6, 7, 10, 14, 21),
    )

```

Additional feature engineering

Further feature engineer using the lagged features to track rolling behaviors using `prepare_features`. With the lagged target features we calculate and/or list:

- Previous week and 2 week target variable values, and their difference.
- Rolling mean and standard deviation for lags 3-7.
- Lag 3 and 7 differences.
- Rolling mean and standard deviation for lags 7 and 14.
- Weekly acceleration behavior, approximated with $(\text{lag}_3 - \text{lag}_5) - (\text{lag}_5 - \text{lag}_7)$ of the target variable.

After these additional transformations, we have the final set of data we will use for training.

```

In [4]: idx = np.arange(len(X))
        X_train = prepare_features(X, idx, X.columns)
        y_train = y

```

Model training

We then initialise the `NHSForecaster` class to train the separate XGBoost model for day N to forecast using `estimated_avoidable_deaths_step_[N]` data. We fix the hyperparameters for all day N forecast XGBoost models to:

```
In [5]: PARAMS = (
        {
            "objective": "reg:squarederror",
            "max_depth": 5,
            "eta": 0.05,
            "subsample": 0.8,
            "colsample_bytree": 0.06,
            "colsample_bylevel": 0.5,
            "colsample_bynode": 0.5,
            "min_child_weight": 25,
            "reg_alpha": 2.0,
            "reg_lambda": 2.0,
            "eval_metric": "rmse",
            "seed": 42,
        },
    )
```

which are also listed in `src/xgboost_model.py`. The aggressive regularisation (`min_child_weight=25`, `colsample_bytree=0.06`) is deliberate given the high feature-to-sample ratio (~3,500 features, ~750 rows), where overfitting is a significant risk. Once all models are trained, they are saved using `joblib` for future forecasting usage or study.

```
In [6]: nhs_model = NHSForecaster()
        trained_models = nhs_model.train_models(X_train, y_train)
        joblib.dump(trained_models, "forecasting_models_notebook.pkl")
```

```
Out[6]: ['forecasting_models_notebook.pkl']
```

Model forecast

We take the most recent values of the training data, and forecast the next 10-days using each of the individually trained XGBoost models.

```
In [7]: todays_values = X_train.iloc[[-1]]
        forecast_df = nhs_model.forecast(todays_values)

        logger.info("Forecast complete.")
        logger.info("-" * 35)
```

```
__main__ INFO Forecast complete.
__main__ INFO -----
__main__ INFO -----
```

```
In [8]: pd.DataFrame(forecast_df)
```

Out [8]:

fore_avoidable_deaths	
day_1_forecast	0.511186
day_2_forecast	0.656094
day_3_forecast	0.566496
day_4_forecast	0.750969
day_5_forecast	0.564114
day_6_forecast	0.305828
day_7_forecast	0.380993
day_8_forecast	0.637338
day_9_forecast	1.130162
day_10_forecast	0.707573

Evaluation data

Model performance is evaluated using walk-forward validation across 173 rolling windows, with MSE computed separately for short-term (D+1 - D+5) and medium-term (D+6 - D+10) horizons. Full results are provided in the attached

`mse_summary.csv` and `pred_matrix.csv`.

```
In [ ]: df = load_and_preprocess_data("../data/turingAI_forecasting_challenge_full")
df = df.reset_index()
df["midday_day"] = pd.to_datetime(df["midday_day"])
logger.info("Data loaded.")

ASSESSMENT_START = pd.Timestamp("2025-10-01", tz="UTC")
df_dev = df[df["midday_day"] < ASSESSMENT_START].copy()
df_assess = df[df["midday_day"] >= ASSESSMENT_START].copy()

HORIZON = 10
n_periods = len(df_assess) - HORIZON + 1

test_data_list = []
forecast_data_list = []

for i in range(n_periods):
    forecast_start = df_assess.iloc[i]["midday_day"]
    cutoff = forecast_start

    # Training data: full development set + assessment rows up to cutoff
    df_train = pd.concat(
        [
            df_dev,
            df_assess[df_assess["midday_day"] <= cutoff],
        ]
    ).copy()
    df_train["midday_day"] = df_train["midday_day"].dt.date

    df_train = weekly_yearly_features(df_train)
```

```

df_train = holiday_features(df_train)

X, y = create_features_and_targets(
    df_train,
    feature_lags=(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 2),
    target_lags=(3, 4, 5, 6, 7, 10, 14, 21),
)
X_train_i = prepare_features(X, np.arange(len(X)), X.columns)

nhs_model = NHSForecaster()
nhs_model.train_models(X_train_i, y)

# Forecast from last training row #
forecasts = nhs_model.forecast(X_train_i.iloc[[-1]])

# Store actuals and predictions as arrays #
actuals = df_assess.iloc[i : i + HORIZON]["estimated_avoidable_deaths"]
test_data_list.append(np.array(actuals))
forecast_data_list.append(np.array(forecasts[:HORIZON].values))

logger.info("Period %d / %d complete.", i + 1, n_periods)

# Compute MSE #
mse_list, summed_mse, mse_1_5_list, summed_mse_1_5, mse_6_10_list, summed_mse_6_10 = compute_mse(
    test_data_list, forecast_data_list
)
logger.info("Overall MSE (1-10): %.4f", summed_mse)
logger.info("Overall MSE (1-5): %.4f", summed_mse_1_5)
logger.info("Overall MSE (6-10): %.4f", summed_mse_6_10)

# Save submission files #

pred_out = pd.DataFrame(forecast_data_list, columns=[f"day_{d + 1}" for d in range(10)])
pred_out.insert(0, "forecast_id", range(1, n_periods + 1))
pred_out.to_csv("pred_matrix.csv", index=False)
logger.info("Saved pred_matrix.csv")

mse_df = pd.DataFrame(
    {
        "forecast_id": range(1, n_periods + 1),
        "mse_1_5": mse_1_5_list,
        "mse_6_10": mse_6_10_list,
    }
)
mse_df.to_csv("mse_summary.csv", index=False)

```

With these forecasts, we compute evaluation metrics:

$$\text{MSE}_{1-5d} = \frac{1}{173 \times 5} \sum_{p=1}^{173} \sum_{d=1}^5 (Y_{p,d} - \hat{Y}_{p,d})^2$$

$$\text{MSE}_{6-10d} = \frac{1}{173 \times 5} \sum_{p=1}^{173} \sum_{d=6}^{10} (Y_{p,d} - \hat{Y}_{p,d})^2$$

where $Y_{p,d}$ is the observed values and $\hat{Y}_{p,d}$ is the forecast for day d within period p .

We find $MSE_{1-5d} = 0.260$ and $MSE_{6-10d} = 0.249$, respectively (medians 0.188 and 0.177, respectively), indicating that forecast skill is well-maintained across the full 10-day horizon. This stability is a direct consequence of the direct forecasting strategy — training one model per horizon day rather than recursively propagating a single-step prediction — which prevents error from compounding across steps. A notable spike is observed around the Christmas and New Year period. This reflects the operational distinctiveness of that period — staffing and demand patterns diverge substantially from the rest of the year — and the limited number of historical Christmas seasons available for training.

```
In [9]: ASSESSMENT_START = pd.Timestamp("2025-10-01", tz="UTC")
HORIZON = 10
n_periods = 131

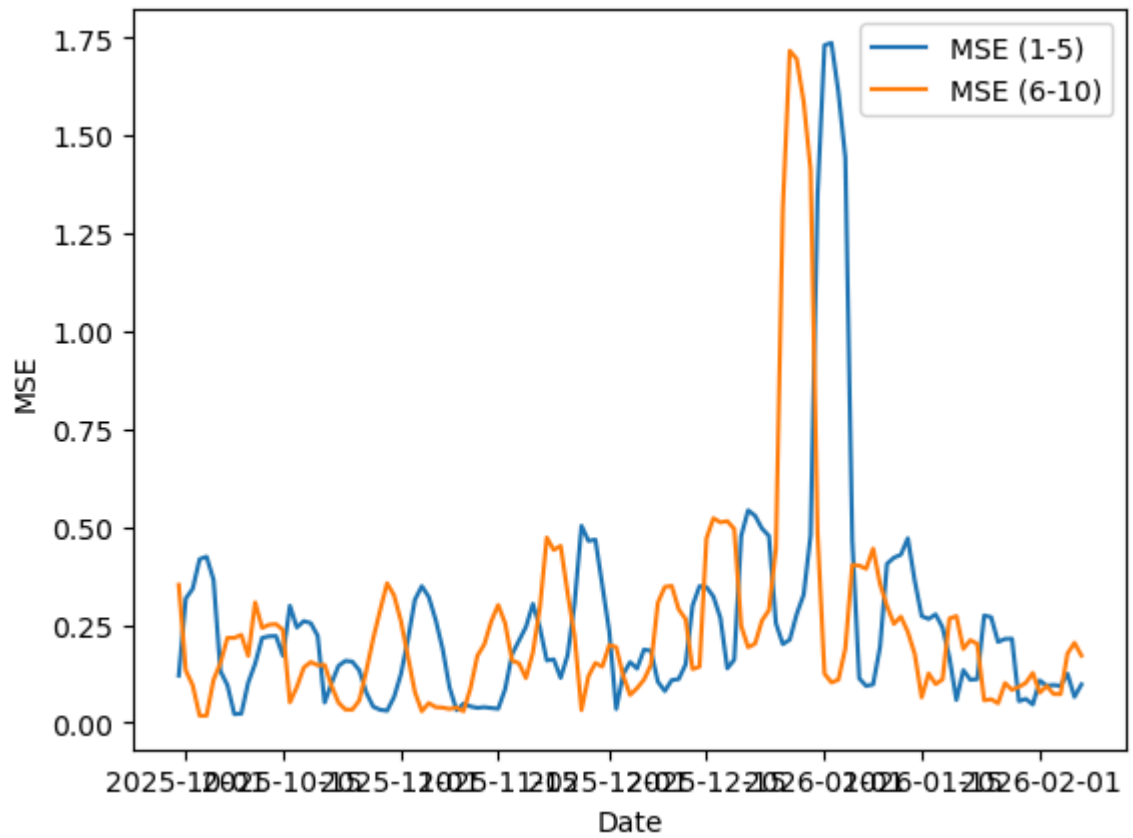
timeline = pd.date_range(
    start=ASSESSMENT_START - pd.tseries.offsets.DateOffset(days=1),
    end=ASSESSMENT_START + pd.tseries.offsets.DateOffset(days=n_periods -
    freq="D",
    )
```

```
In [12]: df = pd.read_csv("mse_summary.csv")
```

```
In [13]: import matplotlib.pyplot as plt

plt.plot(timeline, df["mse_1_5"], label="MSE (1-5)")
plt.plot(timeline, df["mse_6_10"], label="MSE (6-10)")
plt.xlabel("Date")
plt.ylabel("MSE")
plt.legend()
```

```
Out[13]: <matplotlib.legend.Legend at 0x17b124cd0>
```



In []: